CS 182: Lecture 2 Optimization

Note Taker

December 3, 2025

Gemini often includes the announcements too. This is great if we want scribe notes but can be easily commented out if we want the output to be more textbook-esque.

HW0 is due 10:59pm Friday.

This lecture continues our discussion on optimization, building upon the foundations of gradient descent for linear regression. We will explore three key topics:

1. Implicit Regularization of Gradient Descent
2. Stochastic Gradient Descent
3. Momentum

1 Recap: Gradient Descent for Linear Regression

In the previous lecture, we established that Gradient Descent (GD) converges for linear regression problems. The convergence rate is dictated by the learning rate η, which must satisfy the condition:

$$\eta < \frac{1}{\lambda_{\max}(X^T X)}$$

where $\lambda_{\max}(X^T X)$ is the largest eigenvalue of the matrix $X^T X$, which is equivalent to the square of the largest singular value of X.

1.1 Key Takeaway

The geometric properties of the loss function are critical. They directly influence the practical choices of hyperparameters, such as the learning rate. For quadratic loss, the condition number of the Hessian (the ratio of the largest to the smallest singular value) determines the speed of convergence. This intuition generalizes to other functions through concepts like smoothness and strong convexity, which bound a function's curvature.

Details specific to the lecture recording are still being captured by the notes.

2 Implicit Regularization of Gradient Descent

We now explore the connection between Ridge regression, singular values, and the behavior of gradient descent. This will lead us to the concept of *implicit regularization*.

2.1 Ridge Regression and Singular Values

Ridge regression adds an ℓ_2 penalty to the least squares objective, and its solution is given by:

$$\vec{w}_* = (X^T X + \lambda I)^{-1} X^T \vec{y} = X^T (X X^T + \lambda I)^{-1} \vec{y}$$

To understand how Ridge regularization affects the solution, we can analyze it in the basis of the Singular Value Decomposition (SVD) of the data matrix $X = U\Sigma V^T$. After some algebraic manipulation, the solution vector $\vec{w}_*$ can be expressed as a sum over the singular vectors:

$$\vec{w}_* = \sum_{i=1}^{n} \vec{v}_i \left\{ \frac{\sigma_i}{\sigma_i^2 + \lambda} \vec{u}_i^T \vec{y} \right\}$$

This specific version of the output skipped the intermediate steps but we can easily ensure we always keep derivation steps by enforcing that in the prompt.

The term in the curly braces is a scalar coefficient for each singular vector $\vec{v}_i$. The scaling factor $\frac{\sigma_i}{\sigma_i^2+\lambda}$ is crucial.

- If $\lambda \gg \sigma_i$, the scaling factor approaches 0. This means that components of the solution corresponding to small singular values are heavily suppressed.
- If $\sigma_i \gg \lambda$, the scaling factor is approximately $\frac{\sigma_i}{\sigma_i^2} = \frac{1}{\sigma_i}$.

Small singular values correspond to directions of low variance in the data. Ridge regression, therefore, regularizes the solution by shrinking the components along these low-information directions, which helps prevent overfitting.

> "An intriguing recent finding is that neither gradient descent nor stochastic gradient descent moves neutrally to the minimum of the loss function; each exhibits a preference for some solutions over others. This is known as **implicit regularization**." *(Prince, Ch 9, pg 141)*

Prince, Ch 9, pg 141-143 provides a detailed discussion on explicit and implicit regularization.

We get a quote from the textbook. This can either provide an alternate definition, additional background/ context, or interesting observations regarding the topic at hand.

This not only serves to corroborate the notes, but also brings in different perspectives and ways of viewing the same concept, enhancing understanding and making the notes more dynamic and less stand-alone!

2.2 Gradient Descent, Early Stopping, and Weight Decay

The standard gradient descent update for least squares is:

$$\vec{w}_{t+1} = \vec{w}_t - 2\eta X^T (X\vec{w}_t - \vec{y})$$

Notice that the update is always in the span of the data, as it is premultiplied by X^T.

For Ridge regression, the update becomes:

$$\vec{w}_{t+1} = \underbrace{(1 - 2\eta\lambda)}_{\text{weight decay}} \vec{w}_t - 2\eta X^T (X\vec{w}_t - \vec{y})$$

The term $(1 - 2\eta\lambda)$ introduces **weight decay**, multiplicatively shrinking the weights at each step.

To see the implicit regularization of standard GD, we analyze its update in the SVD basis. By defining $\tilde{\vec{w}}_t = V^T \vec{w}_t$, the update for the i-th component becomes:

$$\tilde{w}_{t+1,i} = \tilde{w}_{t,i} - 2\eta\sigma_i[\sigma_i \tilde{w}_{t,i} - \tilde{y}_i]$$

The step size for each component is scaled by its corresponding singular value σ_i. This means GD makes large steps in directions with large singular values and very small steps in directions with small singular values.

We also get direct references to the most relevant pages in the textbook. So if students are confused, want to test their own understanding, or just want to learn more, they can do so easily.

No need to search and wonder if—and where—the textbook contains helpful/ relevant sections, Gemini has already done that distillation.

This behavior leads to the idea of **early stopping**. By stopping the GD algorithm before it has fully converged, we prevent it from making significant progress in the directions of small singular values. This has a similar effect to Ridge regression: it implicitly regularizes the solution by ignoring low-variance directions, which are often associated with fitting noise.

Implicit Regularization Summary

- **Ridge**: When λ is large, the solution does not have large components in directions of small singular values.
- **GD + Early Stopping**: Also does not move significantly (or moves very little) in directions of small singular values.

3 Stochastic Gradient Descent (SGD)

Gradient descent is effective for convex functions, but fitting neural networks involves optimizing over highly non-convex landscapes. Furthermore, GD is computationally expensive for large datasets.

Prince, Ch 6, pg 83-86 introduces SGD and its properties.

Stochastic Gradient Descent (SGD) addresses these issues. The core idea is to approximate the full gradient using a small, randomly sampled subset of the data, called a **mini-batch**.

> "Stochastic gradient descent (SGD) attempts to remedy this problem [getting stuck in local minima] by adding some noise to the gradient at each step. The solution still moves downhill on average, but at any given iteration, the direction chosen is not necessarily in the steepest downhill direction. Indeed, it might not be downhill at all. The SGD algorithm has the possibility of moving temporarily uphill and hence jumping from one "valley" of the loss function to another." *(Prince, Ch 6, pg 85)*

Very minor quotation mark inconsistency here, but this is a problem with latex rather than Gemini.

This time the quote from the textbook foreshadows one core property of SGD: the stochasticity of it allows it to move "uphill", with probability, to escape local minima!

Again, we get a reference on the margins to the exact pages in the textbook that discuss SGD, convenient and accurate.

3.1 The SGD Algorithm

Many standard loss functions are sums over individual data points, $f(\theta) = \frac{1}{n}\sum_{i=1}^{n} f_i(\theta)$. Instead of computing the full gradient $\nabla f(\theta)$, SGD approximates it using a mini-batch B:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla f_i(\theta_t)$$

In expectation, this update is equivalent to a full GD step, since $E[\nabla f_i(\theta)] = \nabla f(\theta)$ if data points are sampled uniformly.

3.1.1 Properties of SGD

- **Less Computationally Expensive**: Gradients are computed on a small subset of data.
- **Noise Injection**: The noisy gradient estimate can help the algorithm escape sharp local minima and saddle points.
- **Convergence**: For convergence, optimization theory suggests using a decreasing learning-rate schedule. However, in practice, SGD can often converge even with a constant step size, especially in overparameterized settings where the loss can go to zero.

Gemini now identifies images that were present in the lecture slides but is unable to "screenshot" it in the latex. Thus the limitation of not having images is still present in this version. I believe this is a rather non-trivial problem, since during lecture we often include screenshots of figures from Prince's textbook which aren't easily scrape-able through a google image search and are often too intricate to be replicated with native latex graphing tools. (We could manually add these screenshots as a supplement to the latex file but that isn't as seamless an experience as an end-to-end LLM pipeline that produces the final result in one go.)

Figure 6.5 from Prince (2023) would be here.

Figure 1: A depiction of Figure 6.5 from Prince (2023). a) Gradient descent with line search. If initialized in the "wrong" valley (e.g., point 2), it will descend toward a local minimum. b) Stochastic gradient descent adds noise to the optimization process, making it possible to escape from the wrong valley (e.g., from point 2) and still reach the global minimum. The figure illustrates how the noisy updates of SGD can be beneficial in non-convex optimization. *Source: Prince 2023.*

4 Momentum

While SGD can help with non-convexity, its path can be erratic. Furthermore, standard GD can be slow when the loss landscape is an elongated valley (i.e., has a poor condition number), as it tends to oscillate across the narrow direction while making slow progress along the valley.

Prince, Ch 6, pg 86-87 explains momentum as a way to smooth the optimization trajectory.

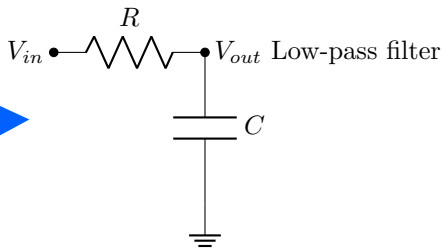
> "A common modification to stochastic gradient descent is to add a momentum term. We update the parameters with a weighted combination of the gradient computed from the current batch and the direction moved in the previous step... The overall effect is a smoother trajectory and reduced oscillatory behavior in valleys." *(Prince, Ch 6, pg 86)*

4.1 Motivation: Smoothing Oscillations

The core idea behind momentum is to smooth out the oscillations in the gradient updates. This can be thought of as **averaging** or **low-pass filtering** the gradient vectors. A naive strategy would be to average the last k gradients, but this requires significant memory.

A more memory-efficient approach is to use an exponentially decaying moving average, which can be implemented using a state-space model. This is analogous to an RC low-pass filter in an electrical circuit, where the capacitor's state (charge) provides memory and smooths out high-frequency fluctuations in the input voltage.

Encouraging Gemini to include packages like tikz and pgfplots now enables recreation of diagrams and (simple) graphs seen during lecture!



If V_{in} is a step of magnitude V, then $V_{out} = V(1 - e^{-t/RC})$.

4.2 The Momentum Update Rule

Momentum introduces a "velocity" or "momentum" vector $\vec{z}$ that accumulates an exponentially decaying average of past gradients. The update rule is:

$$\vec{z}_{k+1} = \beta\vec{z}_k + (1-\beta)\nabla f(\vec{w}_k) \quad \text{where } \vec{z}_0 = 0, \beta \in [0,1)$$
$$\vec{w}_{k+1} = \vec{w}_k - \eta\vec{z}_{k+1}$$

If $\beta = 0$, this reduces to standard gradient descent. For $\beta > 0$, the update $\vec{z}_{k+1}$ is a weighted average of the current gradient and the previous momentum vector. Expanding the recurrence shows that $\vec{z}_{k+1}$ is a sum of all past gradients, with weights that decay exponentially:

$$\vec{z}_3 = \beta^2(1-\beta)\nabla f(\vec{w}_0) + \beta(1-\beta)\nabla f(\vec{w}_1) + (1-\beta)\nabla f(\vec{w}_2)$$

This history of gradients helps the optimizer build "momentum" in consistent directions and dampens oscillations in directions where the gradient sign frequently changes. This often leads to faster convergence.

Another instance of figures being identified but being too complex to recreate using native latex packages (unlike the low-pass filter above, which was captured perfectly).

Figure 6.7 from Prince (2023) would be here.

Figure 2: A depiction of Figure 6.7 from Prince (2023). a) Regular stochastic descent takes a very indirect path toward the minimum. b) With a momentum term, the change at the current step is a weighted combination of the previous change and the gradient computed from the batch. This smooths out the trajectory and increases the speed of convergence.

5 Technical Appendix: SGD Convergence in the Underdetermined Case

We can provably show that SGD converges with a constant step size for an underdetermined linear system, a setting relevant to overparameterized neural networks where the loss can be driven to zero.

Setup: Consider the system $X\vec{w} = \vec{y}$, where $X \in \mathbb{R}^{n \times d}$ with $d > n$ (underdetermined) and X has full row rank. We seek the minimum-norm solution $\vec{w}_* = X^T(XX^T)^{-1}\vec{y}$. We run SGD with batch size 1, starting from $\vec{w}_0 = \vec{0}$.

Let the error vector be $\vec{q}_t = \vec{w}_t - \vec{w}_*$. The problem is equivalent to finding $\vec{q}$ such that $X\vec{q} = \vec{0}$, starting from $\vec{q}_0 = -\vec{w}_*$.

Lyapunov Function: We define a Lyapunov function $L(\vec{z}_t) = ||\tilde{X}\vec{z}_t||^2$, where we have transformed the problem into the SVD basis such that $\tilde{X}\vec{z} = 0$ is the equivalent system in the relevant subspace. Our goal is to show that $E[L(\vec{z}_{t+1})|\vec{z}_t] < L(\vec{z}_t)$, which implies that the loss decreases in expectation at every step. Since the loss is non-negative, it must converge to zero.

The SGD update in the SVD basis is:

$$\vec{z}_{t+1} = \vec{z}_t - 2\eta\tilde{\vec{x}}_{I_t}\tilde{\vec{x}}_{I_t}^T\vec{z}_t$$

where I_t is the index of the data point sampled at time t.

We analyze the change in the loss function:

$$L(\vec{z}_{t+1}) = L(\vec{z}_t) + \underbrace{2\vec{z}_t^T\tilde{X}^T\tilde{X}(\vec{z}_{t+1} - \vec{z}_t)}_{A} + \underbrace{(\vec{z}_{t+1} - \vec{z}_t)^T\tilde{X}^T\tilde{X}(\vec{z}_{t+1} - \vec{z}_t)}_{B}$$

By taking the expectation of terms A and B with respect to the random choice of data point I_t, we can show:

$$E[A|\vec{z}_t] \leq -\frac{4\eta}{n}\sigma_{\min}^2 L(\vec{z}_t)$$
$$E[B|\vec{z}_t] \leq 2\eta^2\sigma_{\max}^2\rho^2 L(\vec{z}_t)$$

where $\sigma_{\min}$ and $\sigma_{\max}$ are the minimum and maximum singular values of $\tilde{X}$, and ρ^2 is an upper bound on $||\vec{x}_{I_t}||^2$.

Putting it all together, we get:

$$E[L(\vec{z}_{t+1})|\vec{z}_t] \leq \left(1 - c_1\eta + c_2\eta^2\right) L(\vec{z}_t)$$

For a sufficiently small learning rate η, the term $(1 - c_1\eta + c_2\eta^2)$ is less than 1, proving that the loss decreases in expectation.

5.1 Takeaways from the Proof

- There is an interesting dependence on the norms of the data points (via ρ^2).
- Decoupling the problem using SVD helps reveal the underlying dynamics.
- SGD shares many properties with GD, such as the updates remaining in the span of the data.

Interesting distinction here. Jameson's original version stopped early because the video transcript didn't get this far. However, the notes, updated after lecture, do.

This version seems to underscore transcribing the most information possible (i.e., preserve as much info from both the slides and the transcript). Thus we see it complete the proof. And we can tell exactly where the recording (i.e., the supporting narrative) ended from the sudden succinctness after the "loss at t+1" expression.